Breaking the Classics: A Cryptography Escape Room

Zineb Elansari

Warm-Up

Question 1: Encrypt (Caesar, $K = 4$)

```
alphabet = "abcdefghijklmnopqrstuvwxyz"

def caesar(text, key):
    result = ""
    for c in text.lower():
        if c in alphabet:
            pos = alphabet.find(c)
            result = result + alphabet[(pos + key) % 26]
        else:
            result = result + c
    return result

print(caesar("CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ", 4))
```

Answer: GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD

Question 2: Decrypt (Caesar, $K = 8$)

```
print(caesar("GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA", -8))
```

Answer: YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS

Question 3: The key is gone (brute force)

```
for k in range(26):
    print(k, caesar("ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP", -k))
```

Answer: for $K = 11$ the text is readable: THE KEY IS UNKNOWN TRY BRUTE FORCE

Room 1: The Letter Wall

Question 1: Encrypt with the substitution alphabet

```
alphabet = "abcdefghijklmnopqrstuvwxyz"
key      = "qmjztgfkpwlsboxncryevhiadu"

def encrypt(text):
    result = ""
    for c in text.lower():
        if c in alphabet:
```

```
            result = result + key[alphabet.find(c)]
        else:
            result = result + c
    return result

print(encrypt("KEEP THE SECRET SAFE"))
```

Answer: LTTN EKT YTJRTE YQGT

Question 2: Decrypt with the same alphabet

```
def decrypt(text):
    result = ""
    for c in text.lower():
        if c in key:
            result = result + alphabet[key.find(c)]
        else:
            result = result + c
    return result

print(decrypt("ERVYE DXVR SXFPJ"))
```

Answer: TRUST YOUR LOGIC

Question 3: Brute force and frequency analysis

Can we try every key? No. A substitution key is a permutation of 26 letters, so there are $26! \approx 4 \times 10^{26}$ keys. This is too many to test one by one.

How can an attacker use letter frequencies? In English some letters are much more common (E, T, A, O...). The attacker counts the letters of the ciphertext. The most frequent ciphertext letter is probably E, the next one T, and so on. This gives part of the key. The rest is found with short words (THE, AND), repeated words and context.

Frequency analysis program:

```
alphabet = "abcdefghijklmnopqrstuvwxyz"

def frequency(text):
    text = text.lower()
    total = 0
    for c in text:
        if c in alphabet:
            total = total + 1
    results = []
    for c in alphabet:
        n = text.count(c)
        if n > 0:
            results.append([n, c])
    results.sort()
    results.reverse()
    for r in results:
        print(r[1], r[0], round(r[0] * 100 / total, 1), "%")
```

Question 4: Break the intercepted message

```
cipher = """EKT YTJRTE BTYYQFT PY KPZZTO PO EKPY NQRQFRQNK.
STEETR GRTCVTOJD ZXTY OXE RTHTQS EKT IKXST LTD,
MVE PE FPHTY Q FXXZ YEQRE.
SXXL GXR JXBBXO IXRZY, RTNTQETZ NQEETROY,
QOZ YKXRE IXRZY.
IKTO DXV RTJXHTR EKT BTYYQFT,
EKT HPFTOTRT LTD KQY GXVR STEETRY."""

frequency(cipher)
```

The output starts with T (17.3%), E (9.6%), R (9.1%), Y (8.1%), X (8.1%), Q (6.1%). The word EKT appears often, so EKT = THE. From the frequencies and the hint, I built this mapping (cipher letter → plain letter) and checked it with a program:

```
m = {"e":"t", "k":"h", "t":"e", "y":"s", "j":"c", "r":"r",
     "b":"m", "q":"a", "f":"g", "p":"i", "z":"d", "o":"n",
     "n":"p", "s":"l", "g":"f", "c":"q", "v":"u", "x":"o",
     "h":"v", "i":"w", "l":"k", "m":"b", "d":"y"}

result = ""
for c in cipher.lower():
    if c in m:
        result = result + m[c]
    else:
        result = result + c
print(result)
```

Answer (plaintext):
THE SECRET MESSAGE IS HIDDEN IN THIS PARAGRAPH.
LETTER FREQUENCY DOES NOT REVEAL THE WHOLE KEY, BUT IT GIVES A GOOD START.
LOOK FOR COMMON WORDS, REPEATED PATTERNS, AND SHORT WORDS.
WHEN YOU RECOVER THE MESSAGE, THE VIGENERE KEY HAS FOUR LETTERS.

Clue for the next room: the Vigenère key has four letters.

Room 2: Break the Vigenère Cipher

```
alphabet = "abcdefghijklmnopqrstuvwxyz"

cipher = "
    VVHWGQUIVYHIRSUPGTWXJSVIPCWIUPHGCIVIVVHPGHWITSDTRSDVUSYITMZLGFHIXSUCVSDQOSPFGFVLQIOHEVHGMSDG
    "
cipher = cipher.lower()

# 1) split the ciphertext into 4 groups
groups = ["", "", "", ""]
for i in range(len(cipher)):
    groups[i % 4] = groups[i % 4] + cipher[i]

# 2) most frequent letter of each group -> assume it is e
key = ""
for g in groups:
    best = ""
```

```
    best_count = 0
    for c in alphabet:
        n = g.count(c)
        if n > best_count:
            best_count = n
            best = c
    shift = (alphabet.find(best) - alphabet.find("e")) % 26
    print("group", best, best_count, "shift", shift, "key letter", alphabet[shift])
    key = key + alphabet[shift]

print("key:", key)

# 3) decrypt the whole message
plain = ""
for i in range(len(cipher)):
    shift = alphabet.find(key[i % 4])
    plain = plain + alphabet[(alphabet.find(cipher[i]) - shift) % 26]
print(plain)
```

Group	Most frequent letter	Shift	Key letter
1	G	2	C
2	S	14	O
3	H	3	D
4	I	4	E

1. Key: CODE

2. Decrypted message: THE SECRET KEEPER LEFT THESE NOTES BECAUSE THE LETTER E APPEARS EVERYWHERE. EVERY TEAM MEMBER SHOULD CHECK EACH COLUMN, SEEK THE MOST FREQUENT SYMBOL AND USE IT TO RECOVER THE KEY. WHEN THE MESSAGE BECOMES CLEAR, ENTER THE ACCESS WORD: ECLIPSE

3. Hidden access word: ECLIPSE